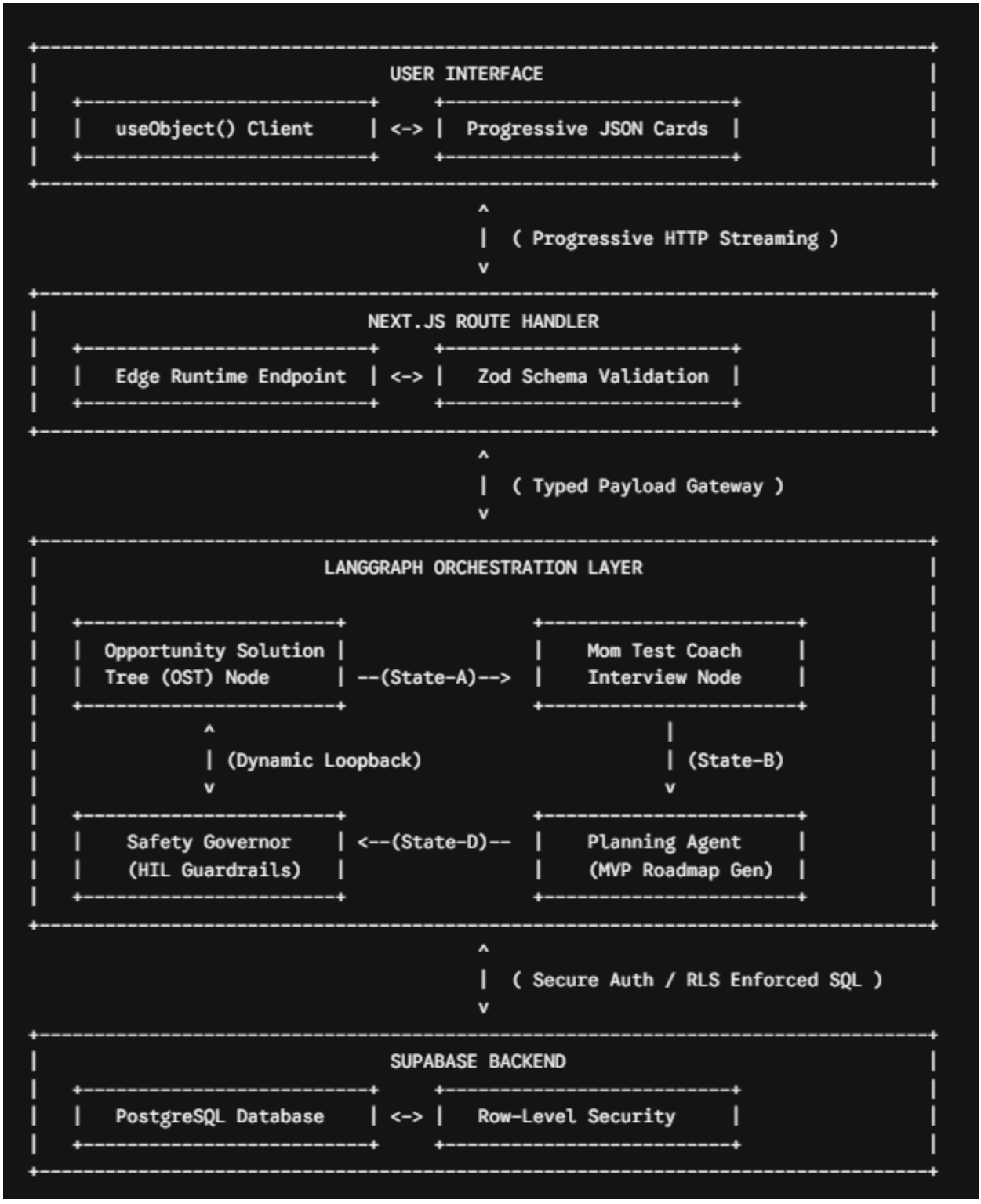
Technical Specification and System Architecture for the Zero-to-One Execution Engine

Product Concept and High-Value Features

The transition from a speculative idea to a viable, structured execution plan is the primary point of failure for early-stage founders, student groups, and career-switchers.[1] Traditional productivity software typically oversimplifies this process through basic linear checklists, or overwhelms users with high-maintenance project management systems.[1] The Zero-to-One Execution Engine is a stateful, progressive product discovery and planning platform designed to convert early-stage concepts into validated, risk-mapped, and sequentially optimized software execution roadmaps.[1]

The system leverages a multi-agent orchestration layer built on LangGraph [4] and a real-time, schema-validated streaming interface powered by the Vercel AI SDK.[5] It is designed to act as an intelligent copilot that systematically reduces the cognitive load of high-stakes product planning.[1]



Opportunity Solution Tree (OST) Deconstruction

The system structures loose concepts using Teresa Torres' Opportunity Solution Tree

framework.[6] Rather than directly prioritizing features, the engine maps customer outcomes to identified opportunities, translates opportunities into candidate product solutions, and links those solutions to non-leading validation tests.[6] This structural mapping ensures that the product is built around real customer outcomes, preventing premature feature convergence.[7]

The Mom Test Validation Coach

Desirability validation must avoid the bias typical of friendly feedback.[10] The engine acts as an interviewer and coach, applying Rob Fitzpatrick's *The Mom Test* principles to help refine the product concept.[10] The system analyzes early user concepts and generates structured user interview scripts designed to collect empirical behavioral facts rather than hypothetical commitments.[10]

This ensures that founders "test the problem before committing to the solution".[12] It maps out target questions, flags leading cues, and evaluates interview transcripts to identify real, monetizable pain points.[12]

Multidimensional Assumption and Experiment Mapping

Following David J. Bland's prioritization framework, the platform identifies and maps business assumptions across four dimensions [9]:

- **Desirability:** Will customers adopt this feature, and does it solve a validated pain point?[11]
- **Viability:** Can the business capture sustainable revenue, and does the solution align with target metrics?[11]
- **Feasibility:** Is the technology constructible within defined financial, operational, and time constraints?[11]
- **Usability:** Can users navigate the interface and complete critical workflows?[11]

Each identified assumption is assigned a coordinate on an Importance versus Evidence grid.[11] Importance ($I \in [0.0, 1.0]$) represents systemic impact if the assumption is false, while Evidence ($E \in [0.0, 1.0]$) represents existing qualitative or quantitative proof.[11] The engine calculates a validation score using the following equation:

$$S_{\text{risk}} = I_{\text{importance}} \times (1.0 - E_{\text{evidence}})$$

Assumptions with high validation scores ($S_{\text{risk}} > 0.60$) are placed in the top-left quadrant of the 2x2 matrix, representing key risks that require immediate testing.[11] For each risk, the system generates a cheap, fast experiment (such as a customer interview, smoke test, concierge flow, or landing page experiment) to validate the assumption before code development begins.[11]

Standardized Model Context Protocol (MCP) Integration

To keep tool calls clean and reliable, the orchestration layer uses the Model Context Protocol.[4] The system interacts with databases, search APIs, and notification systems through three standardized MCP primitives [15]:

- **Tools:** Executable functions the agent calls with validated arguments, such as query_database(sql) or send_onboarding_email(recipient).[15]
- **Resources:** Structured data the agent reads, identified by a URI, such as supabase://tables/projects/schema.[15]
- **Prompts:** Reusable templates the tool server exposes to keep prompts consistent.[15]

Using MCP separates the model's core logic from the actual tool implementations, allowing the team to swap database engines or external APIs without modifying agent code.[15]

Progressive Milestone Decomposition

The system translates verified ideas into sequential, phased milestone roadmaps.[2] Features are arranged by physical dependency to ensure backend foundations (like database schemas and authentication pathways) are completed before building front-end UIs.[2] This structural sequencing helps prevent scope creep, keeping early development focused on a minimum viable loop that creates user habits.[18]

Technical Specifications and System Requirements

The platform is designed around a unified TypeScript system to enable end-to-end type safety across the database, server, and client.[20] This type sharing reduces integration friction, simplifies refactoring, and ensures the codebase remains maintainable for a two-person team.[20]

Functional Requirements

Requirement ID	Module	Specification Description	Empirical Verification Protocol
FR-1.1	Opportunity Solution Tree (OST)	The system must map raw concept descriptions to Teresa Torres' OST framework, outputting explicit Outcome, Opportunity, Solution, and Test nodes.[6]	Submit a loose concept. The system must generate a valid JSON payload containing structured OST nodes with verified associations.[7]

FR-1.2	Mom Test Interviewer	The engine must parse user feedback transcripts and flag leading questions or speculative answers using *The Mom Test* guidelines.[10]	Parse an interview log. The system must output a JSON object highlighting speculative statements and proposing non-leading alternatives.[10]
FR-2.1	Assumption Matrix	The system must generate a 2x2 risk matrix with at least two prioritized assumptions for each dimension (Desirability, Viability, Feasibility, Usability).[11]	Verify that all generated assumptions have non-null categories and non-zero coordinates mapped to the grid.[11]
FR-2.2	Validation Score	The engine must calculate $S_{\text{risk}} = I \times (1.0 -$ for each assumption, flag leap-of-faith risks, and recommend structured experiments.[11]	Run automated unit tests to confirm $S_{\text{risk}} \in$ and verify that recommended experiments are generated for high-risk assumptions.[11]
FR-3.1	MCP Executor	All tool executions (Supabase, Exa Search, Gmail) must be routed through the Model Context Protocol using standard Tool/Resource schemas.[15]	Intercept tool invocations to verify they conform to standard MCP payloads and that schemas are validated using Zod.[15]

FR-4.1	Milestone Generator	The system must convert validated assumptions into a phased 30/60/90-day execution plan with sequential dependencies.[1]	Confirm the generated roadmap is structured into sequential phases where each task has clean, non-circular dependencies.[2]
FR-5.1	Human-in-the-Loop	The state engine must halt before running high-stakes tool actions, requiring manual user approval to continue.[1]	Trigger an external API action. Verify the system enters an awaiting_approval state, writes a transaction log, and pauses execution.[21]

Non-Functional Requirements

Requirement ID	Quality Attribute	Technical Target	Empirical Verification Protocol
NFR-1.1	Latency & Performance	Streaming of structured JSON objects must start within $150\ \text{ms}$ of request receipt.[5]	Deploy on the Next.js Edge Runtime and measure Time-to-First-Byte (TTFB) under a simulated concurrent load.[5]
NFR-2.1	Schema Integrity	Every model output must conform to a strict Zod schema contract before reaching application state.[21]	Run a test suite with invalid schemas. Confirm the API route rejects malformed payloads and handles validation

			errors gracefully.[21]
NFR-2.2	End-to-End Typing	The application must maintain 100% type safety across Supabase schemas, edge functions, and client states.[20]	Run tsc --noEmit and verify that any schema changes in Supabase automatically trigger compile-time type check errors in client code.[20]
NFR-3.1	Database Security	Every user row query must be validated at the database engine layer, preventing unauthorized cross-tenant data access.[24]	Attempt to query rows using a mismatching auth session. Verify the database rejects the query with an RLS authorization error.[24]
NFR-4.1	Execution Safety	The multi-agent orchestrator must limit runaway loops and set explicit token spend caps per user session.[16]	Configure the orchestrator with stopWhen: stepCountIs(5). Run a circular tool call loop to verify the system halts once the step cap is met.[21]

Architectural Constraints

Building a high-performance system within a fast-moving, competitive timeline requires the engineering team to design the architecture around specific technological, runtime, and security constraints.[1]

Operational Timeframe

The system must be built, tested, and deployed within a strict 7-day window.[1] This schedule requires the team to use pre-built UI components and managed backend services (like Supabase Auth, PostgreSQL, and storage buckets) to avoid building standard boilerplate from scratch.[18]

Compute Budgets and Loop Prevention

Autonomous multi-agent systems are prone to infinite execution loops when models recursively call tools to resolve errors, which can quickly drain API credits.[16] To mitigate this, the systems architecture enforces strict limits:

- **Step Cap:** Every graph traversal has a hard limit of stopWhen: stepCountIs(5).[21]
- **Edge Timeout:** API routes must complete execution within a 30-second window before Vercel terminates the connection.[5]
- **Fallback Paths:** If an API call fails or times out, the system must trigger backoff retries and default to user clarification.[21]

Data Protection and Compliance

User information must be isolated at the database engine layer to prevent data leakage in shared workspaces.[24] All tables use PostgreSQL Row-Level Security (RLS) with policies linked to Supabase Auth UUIDs.[24] Tool input schemas must also redact sensitive parameters or Personally Identifiable Information (PII) before writing execution logs to storage.[21]

Edge Runtime Compatibility

To minimize cold starts, all streaming endpoints run on the Next.js Edge Runtime.[5] This lightweight environment does not support standard Node.js filesystem operations (fs) or un-optimized native packages.[5] All third-party libraries must be edge-compatible, and external integrations must route through the Vercel AI Gateway for efficient traffic management.[5]

Technical Architecture

The architecture is split into a **Next.js 15 Client Interface** leveraging the useObject hook for progressive rendering [33], a **LangGraph Multi-Agent Orchestration Layer** running on the Edge Runtime [4], and a **Supabase PostgreSQL Database** configured with security guardrails.[25]

Multi-Agent Stateful Orchestration (LangGraph Node Mechanics)

The graph maintains user state in a central thread, allowing specialized nodes to read, process, and update the execution plan.[34]

1. **Clarification Agent (OST Node):** Parses the raw concept, identifies core user jobs using the JTBD framework, and maps outcomes to opportunities.[6]
2. **Mom Test Coach Node:** Generates behavior-focused questions based on Fitzpatrick's rules, analyzes feedback logs, and identifies target opportunities.[10]
3. **Risk Assessment Agent (Assumption Grid Node):** Generates prioritized assumptions, calculates validation scores (S_{risk}), and identifies leap-of-faith risks.[11]
4. **Planning Agent (Roadmap Node):** Decomposes concepts into sequential development

tasks with clear, logical dependencies.[2]

5. **Safety Governor Node:** Evaluates the proposed plan, checks for compliance risks, and calculates a system confidence index.[1] If the confidence index is below 0.70, the graph loops back to the Clarification Agent for refinement.[21] If confidence is high, the system saves the state, triggers the Human-in-the-Loop gateway, and streams the structured payload to the client.[1]

End-to-End Type Contract (Zod Schema)

To ensure type safety, the application uses shared Zod validation schemas.[21] This schema defines the structural contract for all generated execution plans [21]:

```
TypeScript
import { z } from 'zod'

export const OstNodeSchema = z.object({
  id: z.string().uuid(),
  type: z.enum(['outcome', 'opportunity', 'solution', 'test']),
  title: z.string().describe('Clear, short description of the opportunity solution tree node.'),
  parentId: z.string().uuid().nullable().describe('Valid reference linking back to parental outcome hierarchy.'),
});

export const MomTestPromptSchema = z.object({
  targetHypothesis: z.string().describe('The core assumption being tested.'),
  nonLeadingQuestions: z.array(z.string()).min(3).max(5).describe('Questions targeting past behaviors.'),
  antiPatternsToAvoid: z.array(z.string()).describe('Identified leading or hypothetical statements to avoid.'),
});

export const JtbdStorySchema = z.object({
  role: z.string().describe('The primary actor or target persona.'),
  situation: z.string().describe('The triggering context: When this event happens.'),
  motivation: z.string().describe('The core motivation: I want to perform this action.'),
  outcome: z.string().describe('The emotional, functional, or social benefit: So that I can achieve this.'),
  dimension: z.enum(['functional', 'emotional', 'social']).describe('The classification of user need.'),
```

```
});

export const AssumptionSchema = z.object({
  id: z.string().uuid(),
  category: z.enum(['desirability', 'viability', 'feasibility', 'usability']),
  statement: z.string().describe('Empirical assumption formulated as a factual truth.'),
  importance: z.number().min(0.0).max(1.0).describe('Impact of assumption failure (0 = minimal, 1 = fatal).'),
  evidence: z.number().min(0.0).max(1.0).describe('Level of existing empirical proof (0 = assumption, 1 = validated).'),
  validationScore: z.number().min(0.0).max(1.0).describe('Calculated validation score: Importance * (1.0 - Evidence).'),
  recommendedExperiment: z.string().describe('Fastest, lowest-cost experiment to gather validation evidence.'),
});

export const RoadmapTaskSchema = z.object({
  id: z.string(),
  title: z.string().describe('Direct action-oriented task title.'),
  durationDays: z.number().min(1).describe('Estimated days to build.'),
  dependencies: z.array(z.string()).describe('Identified tasks that must complete first.'),
  complexity: z.enum(['low', 'medium', 'high']),
  alternativeApproach: z.string().optional().describe('Alternative validation pathway if technical blockers arise.'),
});

export const MilestoneSchema = z.object({
  phase: z.string().describe('Phase identifier (e.g., Phase 1: Core Loop Validation).'),
  objective: z.string().describe('The physical outcome of this milestone.'),
  tasks: z.array(RoadmapTaskSchema),
});

export const SystemGovernanceSchema = z.object({
  confidenceIndex: z.number().min(0.0).max(1.0).describe('Overall systems validation confidence.'),
  requiresHumanApproval: z.boolean().describe('Triggers a physical UI block requiring manual confirmation.'),
  governanceWarning: z.string().optional().describe('Warning detailing planning assumptions or key limitations.'),
});

export const MasterExecutionPlanSchema = z.object({
  conceptName: z.string().describe('The refined, action-oriented name of the concept.'),
```

```
  ostFramework: z.array(OstNodeSchema),
  momTestValidation: MomTestPromptSchema,
  jtbdFramework: z.array(JtbdStorySchema),
  prioritizedAssumptions: z.array(AssumptionSchema),
  milestones: z.array(MilestoneSchema),
  governance: SystemGovernanceSchema,
});

export type MasterExecutionPlan = z.infer<typeof MasterExecutionPlanSchema>;
```

Next.js Edge Routing and Progressive Client Streaming

The communication pipeline uses the Vercel AI SDK to stream structured payloads directly from the server to the client.[5] The backend API route processes the incoming prompt using the streamObject function, which serializes the output into an incremental stream of JSON tokens.[5]
By applying the anthropic-beta fine-grained tool streaming header, the system streams complex nested arrays progressively, token-by-token.[29] This approach allows the frontend client to render the schema data as soon as the first fields are received.[33]

```
TypeScript
// app/api/builder/route.ts
import { anthropic } from '@ai-sdk/anthropic'
import { streamObject } from 'ai'
import { MasterExecutionPlanSchema } from '@/schemas/builder'

export const runtime = 'edge'
export const maxDuration = 30

export async function POST(req: Request) {
  try {
    const { conceptPrompt, sessionUserId } = await req.json();

    const result = streamObject({
      model: anthropic('claude-sonnet-4-20250514'),
      schema: MasterExecutionPlanSchema,
      schemaName: 'MasterExecutionPlan',
```

```
    schemaDescription: 'Decoupled architectural execution and risk mapping blueprint.',
    prompt: `Generate a Zero-to-One execution plan based on this concept description: "${conceptPrompt}".`,
    system: `You are an expert AI-Native Systems Architect. Deconstruct the concept into structural JTBD statements, identify assumptions, calculate validation scores, phase development tasks logically, and flag governance markers.`,
    headers: {
      'anthropic-beta': 'fine-grained-tool-streaming-2025-05-14'
    }
  });

  return result.toTextStreamResponse();
} catch (error) {
  return new Response(JSON.stringify({ error: 'Failed to process execution stream' }), {
    status: 500,
    headers: { 'Content-Type': 'application/json' }
  });
}
}
```

On the frontend, the useObject client hook consumes this JSON stream progressively.[33] As tokens arrive, the user interface updates dynamically, rendering partial cards, assumption grids, and roadmaps without forcing the user to wait for the complete generation loop to finish.[33]

TypeScript

```
// app/builder/client.tsx
'use client'

import { experimental_useObject as useObject } from '@ai-sdk/react';
import { MasterExecutionPlanSchema } from '@/schemas/builder';
import { Button } from '@/components/ui/button';
import { AlertCircle, ShieldAlert } from 'lucide-react';

export default function ZeroToOneBuilder() {
  const { object, submit, isLoading, stop } = useObject({
    api: '/api/builder',
    schema: MasterExecutionPlanSchema,
  });
```

```
  return
    <div className="container mx-auto p-6 max-w-5xl space-y-8">
      <div className="flex justify-between items-center">
        <h1 className="text-3xl font-extrabold tracking-tight">Zero-to-One Builder Workspace</h1>
        <Button
          onClick={() => submit({ conceptPrompt: "AI-based inventory manager for small local
businesses." })}
          disabled={isLoading}
        >
          {isLoading? 'Streaming Architecture...' : 'Generate Build Plan'}
        </Button>
      </div>

      {isLoading && (
        <div className="flex items-center gap-2 text-blue-600 bg-blue-50 p-4 rounded-lg">
          <AlertCircle className="animate-spin" />
          <span>Generating architecture. Rendering progressive interface segments...</span>
          <Button size="sm" variant="outline" onClick={() => stop()}>Halt Stream</Button>
        </div>
      )}

      {/* Progressive Governance Safeguard Alert */}
      {object?.governance?.governanceWarning && (
        <div className="p-4 bg-amber-50 border-l-4 border-amber-500 rounded-lg flex items-start
gap-3">
          <ShieldAlert className="text-amber-600 mt-1 flex-shrink-0" />
          <div>
            <h4 className="font-bold text-amber-900">Responsible AI Safeguard Advisory</h4>
            <p className="text-amber-800 text-sm">{object.governance.governanceWarning}</p>
          </div>
        </div>
      )}

      {/* Rendering System Components progressively using optional chaining */}
      <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
        {/* JTBD Display Card */}
        {object?.conceptName && (
          <div className="p-6 bg-white border rounded-xl shadow-sm space-y-4">
            <h3 className="text-xl font-bold tracking-tight">Structured Concept:
{object.conceptName}</h3>
            <div className="space-y-3">
              {object.jtbdFramework?.map((job, idx) => (
                <div key={idx} className="p-3 bg-slate-50 rounded-lg text-sm">
                  <span className="font-bold uppercase text-[10px] text-slate-500 px-1.5 py-0.5
bg-slate-200 rounded">
                    {job?.dimension}
```

```
                </span>
                <p className="mt-1 text-slate-700">
                  When <strong>{job?.situation}</strong>, I want to <strong>{job?.motivation}</strong>, so
that I can <strong>{job?.outcome}</strong>.
                </p>
              </div>
            ))}
          </div>
        </div>
      )}

      {/* Assumptions Map */}
      {object?.prioritizedAssumptions && (
        <div className="p-6 bg-white border rounded-xl shadow-sm space-y-4">
          <h3 className="text-xl font-bold tracking-tight">Assumption Prioritization Map</h3>
          <div className="space-y-2">
            {object.prioritizedAssumptions.map((assump, idx) => (
              <div key={idx} className="p-3 border rounded-lg flex justify-between items-center
text-sm">
                <div>
                  <p className="font-medium text-slate-800">{assump?.statement}</p>
                  <p className="text-xs text-slate-400">Score: {assump?.validationScore?.toFixed(2)}</p>
                </div>
                <span className={`px-2 py-0.5 rounded text-[10px] font-bold ${
                  (assump?.validationScore?? 0) > 0.6? 'bg-red-100 text-red-700' : 'bg-green-100
text-green-700'
                }`}>
                  {(assump?.validationScore?? 0) > 0.6? 'RISK: TEST ASAP' : 'MONITOR'}
                </span>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  );
}
```

Persisted Database Structure and Security (Supabase Schema)

To ensure reliable performance, the relational database structure must reflect the application models.[18] The schema leverages foreign key constraints to link components together, while PostgreSQL Row-Level Security (RLS) policies enforce security barriers at the database engine layer.[24]

```sql
-- Enable UUID Extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 1. Profiles Table
CREATE TABLE public.profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT UNIQUE NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow users to view their own profile"
  ON public.profiles FOR SELECT
  USING (auth.uid() = id);

CREATE POLICY "Allow users to update their own profile"
  ON public.profiles FOR UPDATE
  USING (auth.uid() = id);

-- 2. Concepts Table
CREATE TABLE public.concepts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  name TEXT NOT NULL,
  raw_input TEXT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.concepts ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Allow users access to their own concepts"
  ON public.concepts FOR ALL
  USING (auth.uid() = user_id);

-- 3. OST Nodes Table
CREATE TABLE public.ost_nodes (
```

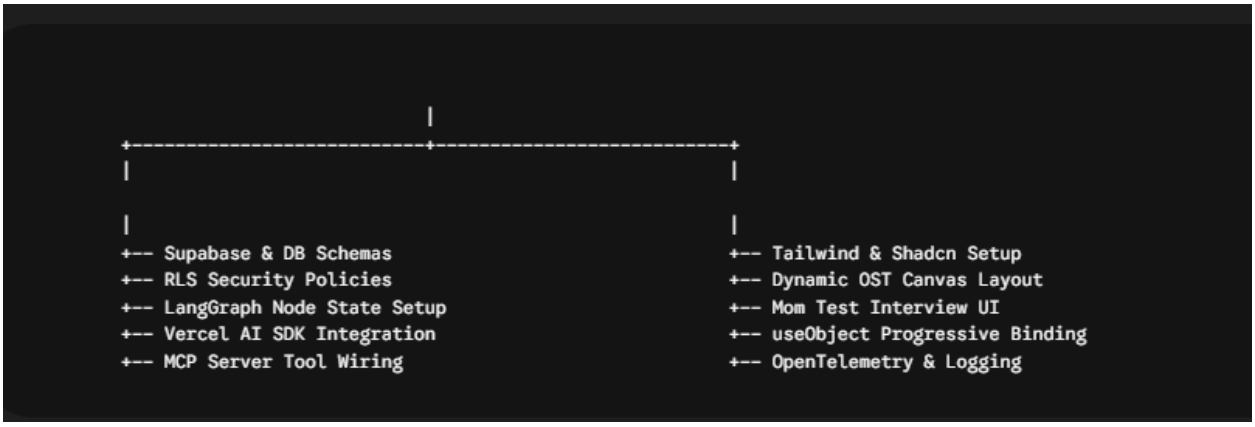
```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    concept_id UUID REFERENCES public.concepts(id) ON DELETE CASCADE NOT NULL,
    type TEXT CHECK (type IN ('outcome', 'opportunity', 'solution', 'test')) NOT NULL,
    title TEXT NOT NULL,
    parent_id UUID REFERENCES public.ost_nodes(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.ost_nodes ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Enforce user concept isolation for OST nodes"
    ON public.ost_nodes FOR ALL
    USING (EXISTS (
        SELECT 1 FROM public.concepts
        WHERE concepts.id = ost_nodes.concept_id AND concepts.user_id = auth.uid()
    ));

-- 4. Assumptions Table
CREATE TABLE public.assumptions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    concept_id UUID REFERENCES public.concepts(id) ON DELETE CASCADE NOT NULL,
    category TEXT CHECK (category IN ('desirability', 'viability', 'feasibility', 'usability')) NOT NULL,
    statement TEXT NOT NULL,
    importance REAL NOT NULL CHECK (importance >= 0.0 AND importance <= 1.0),
    evidence REAL NOT NULL CHECK (evidence >= 0.0 AND evidence <= 1.0),
    validation_score REAL NOT NULL,
    recommended_experiment TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.assumptions ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Enforce user concept isolation for assumptions"
    ON public.assumptions FOR ALL
    USING (EXISTS (
        SELECT 1 FROM public.concepts
        WHERE concepts.id = assumptions.concept_id AND concepts.user_id = auth.uid()
    ));

-- 5. Milestones Table
CREATE TABLE public.milestones (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    concept_id UUID REFERENCES public.concepts(id) ON DELETE CASCADE NOT NULL,
```

```
    phase TEXT NOT NULL
    objective TEXT NOT NULL
    tasks JSONB NOT NULL
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.milestones ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Enforce user concept isolation for milestones"
    ON public.milestones FOR ALL
    USING (EXISTS (
        SELECT 1 FROM public.concepts
        WHERE concepts.id = milestones.concept_id AND concepts.user_id = auth.uid()
    ));

-- 6. Telemetry and Audit Logs Table
CREATE TABLE public.audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    concept_id UUID REFERENCES public.concepts(id) ON DELETE CASCADE NOT NULL,
    steps_executed INTEGER NOT NULL,
    tool_calls JSONB NOT NULL,
    final_status TEXT NOT NULL,
    governance_flag BOOLEAN DEFAULT false NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

ALTER TABLE public.audit_logs ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Enforce user concept isolation for audit logs"
    ON public.audit_logs FOR ALL
    USING (EXISTS (
        SELECT 1 FROM public.concepts
        WHERE concepts.id = audit_logs.concept_id AND concepts.user_id = auth.uid()
    ));
```

Technical Labor Allocation and Development Phases

To ensure clear ownership and avoid integration issues, the workload is split 50/50 between two engineers.[1] Developer A manages the database, multi-agent logic, and backend API routes, while Developer B builds the client components, progressive streaming interface, and telemetry tracking.[20]

Work Breakdown Structure (50/50 Task Division)



Component Group	Developer A (Backend, AI, & Infrastructure)	Developer B (Frontend, Client, & UI)	Shared Validation Gateway
Foundation & Database	Sets up public schemas, configures auth routing, and writes SQL Row-Level Security policies.[24]	Configures Tailwind CSS and scaffolds Next.js App Router folders.[20]	**Schema Validation:** Verify compile-time type safety across the database and frontend models.[20]
Core AI State Engine	Configures the LangGraph multi-agent state nodes and maps the state transition logic.[4]	Designs the OST visual interface and maps hierarchical node cards dynamically.[6]	**Graph Traversal:** Ensure state transitions complete without loops, verified in a local test suite.[34]
Streaming Pipeline	Implements Next.js Edge API routes and connects Zod validators to streamObject.[5]	Mounts the Vercel AI SDK useObject hook and binds it to client states.[33]	**Integration Handshake:** Verify the client successfully reads and renders progressive JSON streams.[33]
Validation Workspace	Integrates Exa Search and other external tools via	Builds the 2x2 assumption prioritization layout	**Tool Execution:** Verify the model successfully calls

	standard MCP server primitives.[15]	using SVG and Framer Motion.[11]	external search tools through the MCP schema.[15]
Safety & Telemetry	Implements the safety governor, sets the loop ceiling, and builds the validation gates.[21]	Configures telemetry logging using Langfuse and registers Sentry tracking spans.[42]	**Security Sign-off:** Run verification checks to confirm RLS blocks unauthorized queries at the database layer.[24]

Conclusions and Technical Recommendations

To maximize performance and keep development on track, the team should implement the following engineering guidelines [1]:

Use Progressive Array Streaming

Generating large, nested JSON plans can introduce latency when streaming over standard connection profiles.[29] The team must include the anthropic-beta: fine-grained-tool-streaming-2025-05-14 header on Anthropic routes.[29] This forces the model to stream array items progressively as they are parsed, allowing the client to render them field-by-field without wait times.[37]

Enforce strict loop limits via the Safety Governor Node

To prevent runaway model loops, developers must enforce strict boundaries at the orchestration layer.[21] The system must set a hard ceiling of stopWhen: stepCountIs(5) on all LangGraph nodes.[21] Any workflow exceeding this limit must be terminated, write an incident log to the Supabase database, and request manual input.[21]

Leverage PostgreSQL RLS over API Verification

Relying solely on middleware to validate tenant boundaries increases the risk of manual routing bugs and security leaks.[24] Enabling Row-Level Security on PostgreSQL ensures that query restrictions are enforced directly at the database engine layer.[24] This creates a secure, zero-trust backend that isolates tenant workspaces even if the API routing layer fails.[24]